增强版触摸绘画应用程序

增强版触摸绘画应用程序
- 例程效果
- 代码讲解
  - 1. 导入必要的模块
  - 2. 初始化触摸传感器和屏幕常量
  - 3. 定义 `DrawingApp` 类
    - 3.1 初始化绘画状态和UI参数
    - 3.2 创建显示图像并初始化界面
  - 4. 用户界面绘制方法
    - 4.1 清空画布
    - 4.2 绘制用户界面
  - 5. 触摸事件处理逻辑
    - 5.1 主触摸处理方法
    - 5.2 处理绘画操作
    - 5.3 处理UI交互
  - 6. 更新显示
  - 7. 主程序逻辑
    - 7.1 主循环：读取并处理触摸数据
    - 7.2 异常处理与资源释放
  - 8. 主程序入口

例程效果

本节例程源码文件名为 `enhanced_touch_drawing.py`。

连接IDE，在IDE中运行代码，你将看到一个带有画布和工具栏的绘画应用程序界面。可以通过触摸屏幕在画布区域绘制图形，选择不同的颜色和画笔粗细，还可以清空画布。工具栏包含颜色选择、画笔大小调节以及控制按钮（如清屏）。



代码讲解

1. 导入必要的模块

```
import time, os, sys
from media.display import *
from media.media import *
from machine import TOUCH
```

- `time`, `os`, `sys`：系统相关的模块，用于时间控制、操作系统功能和系统操作。
- `media.display` 和 `media.media`：用于屏幕显示和媒体资源管理。
- `machine.TOUCH`：用于初始化和操作触摸传感器。

2. 初始化触摸传感器和屏幕常量

```
tp = TOUCH(0)
DISPLAY_WIDTH = 640
DISPLAY_HEIGHT = 480
CANVAS_WIDTH = 480
CANVAS_HEIGHT = 480
TOOLBAR_WIDTH = 160
```

- `TOUCH(0)`：在引脚 0 上初始化触摸传感器，`tp` 用于后续读取触摸数据。
- `DISPLAY_WIDTH` 和 `DISPLAY_HEIGHT`：定义屏幕总分辨率为 640x480。
- `CANVAS_WIDTH` 和 `CANVAS_HEIGHT`：定义画布区域的分辨率为 480x480。
- `TOOLBAR_WIDTH`：定义工具栏宽度为 160 像素。

3. 定义 `DrawingApp` 类

```
class DrawingApp:
    def __init__(self):
        """初始化绘画应用程序"""
```

- `DrawingApp` 类是整个绘画应用程序的核心，负责管理绘画状态、用户界面绘制和触摸事件的处理。

3.1 初始化绘画状态和UI参数

```
self.current_color = (0, 0, 0)
self.current_brush_size = 5
self.is_drawing = False
self.last_x = None
self.last_y = None
```

- `current_color`：当前选中的绘画颜色，默认黑色。
- `current_brush_size`：当前画笔大小，默认 5 像素。
- `is_drawing`：布尔值，记录是否正在绘画。
- `last_x` 和 `last_y`：记录上一个触摸点的坐标，用于绘制连续线条。

此外，代码还定义了颜色调色板、画笔大小选项以及UI布局参数（如颜色选择区域、画笔大小选择区域和控制按钮区域的位置和尺寸）。

3.2 创建显示图像并初始化界面

```
self.display_image = image.Image(DISPLAY_WIDTH, DISPLAY_HEIGHT, image.RGB888)
self.clear_canvas()
self.draw_ui()
```

- 创建一个 RGB888 格式的图像对象 `display_image`，用于显示整个界面和绘画内容。
- 调用 `clear_canvas()` 清空画布并设置白色背景。
- 调用 `draw_ui()` 绘制用户界面（工具栏、颜色选择、画笔大小选择和控制按钮）。

4. 用户界面绘制方法

4.1 清空画布

```
def clear_canvas(self):
    """清空画布"""
    self.display_image.clear()
    self.display_image.draw_rectangle(0, 0, CANVAS_WIDTH, CANVAS_HEIGHT, color=
(255, 255, 255), fill=True)
    self.display_image.draw_rectangle(CANVAS_WIDTH, 0, TOOLBAR_WIDTH,
CANVAS_HEIGHT, color=(240, 240, 240), fill=True)
    self.display_image.draw_line(CANVAS_WIDTH, 0, CANVAS_WIDTH, CANVAS_HEIGHT,
color=(200, 200, 200), thickness=2)
```

- 清空整个显示图像。
- 绘制白色画布背景和灰色工具栏背景。
- 在画布和工具栏之间绘制分隔线。

4.2 绘制用户界面

```
def draw_ui(self):
    """绘制用户界面"""
```

- `draw_ui()` 调用多个子方法绘制工具栏、颜色调色板、画笔大小选择和控制按钮。
- 具体方法包括 `draw_color_palette()`（绘制颜色选择）、`draw_brush_size_palette()`（绘制画笔大小选择）、`draw_control_buttons()`（绘制控制按钮）和 `draw_status_info()`（绘制当前状态信息）。

5. 触摸事件处理逻辑

5.1 主触摸处理方法

```
def handle_touch(self, x, y, event):
    """处理触摸事件"""
    if x < CANVAS_WIDTH:
        self.handle_drawing(x, y, event)
    else:
        self.handle_ui_interaction(x, y, event)
```

- 根据触摸点的位置判断是在画布区域（`x < CANVAS_WIDTH`）还是工具栏区域，并分别调用 `handle_drawing()` 或 `handle_ui_interaction()` 处理。

5.2 处理绘画操作

```
def handle_drawing(self, x, y, event):
    """处理绘画操作"""
    if event == TOUCH.EVENT_DOWN:
        self.is_drawing = True
        self.last_x = x
        self.last_y = y
        self.display_image.draw_circle(x, y, self.current_brush_size//2,
color=self.current_color, fill=True)
    elif event == TOUCH.EVENT_MOVE and self.is_drawing:
        if self.last_x is not None and self.last_y is not None:
            self.display_image.draw_line(self.last_x, self.last_y, x, y,
color=self.current_color, thickness=self.current_brush_size)
        self.last_x = x
        self.last_y = y
    elif event == TOUCH.EVENT_UP:
        self.is_drawing = False
        self.last_x = None
        self.last_y = None
```

- 触摸按下事件（`EVENT_DOWN`）：开始绘画，记录起点并绘制一个圆点。
- 触摸移动事件（`EVENT_MOVE`）：如果正在绘画，连接上一个点和当前点绘制线条。
- 触摸抬起事件（`EVENT_UP`）：结束绘画，重置上一个点的坐标。

5.3 处理UI交互

```
def handle_ui_interaction(self, x, y, event):
    """处理UI交互"""
    if event != TOUCH.EVENT_DOWN:
        return
    if self.color_palette_y <= y <= self.color_palette_y + 60:
        self.handle_color_selection(x, y)
    elif self.brush_palette_y <= y <= self.brush_palette_y +
len(self.brush_sizes) * self.brush_spacing:
        self.handle_brush_size_selection(x, y)
    elif self.control_buttons_y <= y <= self.control_buttons_y + 80:
        self.handle_control_buttons(x, y)
```

- 仅处理触摸按下事件（`EVENT_DOWN`）。
- 根据触摸点 Y 坐标判断是在颜色选择区域、画笔大小选择区域还是控制按钮区域，并调用相应方法处理。

6. 更新显示

```
def update_display(self):
    """更新显示 - 简化版本，避免像素级操作"""
    Display.show_image(self.display_image)
```

- 使用 `Display.show_image()` 更新屏幕显示当前图像，避免复杂的像素级操作。

7. 主程序逻辑

```
def main():
    """主程序"""
    print("启动增强版触摸绘画程序...")
    Display.init(Display.ST7701, width=DISPLAY_WIDTH, height=DISPLAY_HEIGHT,
to_ide=True)
    MediaManager.init()
    app = DrawingApp()
```

- 初始化显示器和媒体管理器。
- 创建 `DrawingApp` 实例，启动绘画应用程序。

7.1 主循环：读取并处理触摸数据

```
while True:
    os.exitpoint()
    points = tp.read(1)
    if len(points) > 0:
        pt = points[0]
        app.handle_touch(pt.x, pt.y, pt.event)
        app.update_display()
    time.sleep(0.02)
```

- 持续读取触摸数据。
- 如果有触摸事件，调用 `handle_touch()` 处理，并更新显示。
- 每 20 毫秒更新一次（约 50 FPS）。

7.2 异常处理与资源释放

```
except KeyboardInterrupt:
    print("用户停止程序")
except Exception as e:
    print(f"程序异常: {e}")
finally:
    Display.deinit()
    os.exitpoint(os.EXITPOINT_ENABLE_SLEEP)
    time.sleep_ms(100)
    MediaManager.deinit()
```

- 捕获用户中断（`KeyboardInterrupt`）和其他异常（`Exception`），并打印相关信息。
- 反初始化显示器，设置退出点为睡眠模式，并释放媒体资源。

8. 主程序入口

```
if __name__ == "__main__":
    os.exitpoint(os.EXITPOINT_ENABLE)
    main()
```